

About the ICEPOT Framework

A weak prompt gives the AI a task but no information about the product, rules, expected format or quality bar, so the output is generic and incomplete. ICEPOT fixes this by giving every prompt six building blocks:

Element	Purpose
I – Instruction	The exact task the AI must perform.
C – Context	Product domain, business rules, limits, platforms and constraints.
E – Example	A sample of the expected output so the AI copies the structure and depth.
P – Persona	The role/expertise the AI should adopt.
O – Output Format	Columns, grouping and the minimum number of test cases.
T – Tone	The writing style of the output.

Scenario 1 — Search Functionality

✗ Weak Prompt: *"Test the search feature."*

What's missing: No product type (e-commerce, hospital), no filter/sort behavior, no empty/invalid input scenarios, no performance angle.

Improved Prompt (ICEPOT)

You are a Senior QA Engineer with 8+ years of experience testing high-traffic e-commerce platforms, skilled in functional, negative and performance testing. Generate a comprehensive set of functional, negative, boundary and performance test cases for the product search feature, covering keyword search, filters, sorting, auto-suggest and pagination. Context: The application is a B2C e-commerce web portal (similar to Amazon/Flipkart) with a catalogue of ~50,000 products. Search supports keyword, partial match and category search. Filters: Category, Brand, Price range, Rating (1–5★), Availability. Sort: Relevance, Price Low→High, Price High→Low, Newest, Customer Rating. Results show 20 items per page. Auto-suggest appears after 3 characters. Expected response time: results within 2 seconds for 500 concurrent users. Empty search, special characters (@#\$%), SQL/script strings (' OR 1=1, <script>), 255+ characters and leading/trailing spaces must be handled gracefully with a friendly message. Here is an example of the expected test case: TC_SRCH_001 | Search with valid keyword | Pre-condition: user on Home page | Steps: 1. Enter "running shoes" 2. Press Enter | Test Data: running shoes | Expected: Results page lists only shoe products matching the keyword, result count displayed, 20 items per page | Priority: High | Type: Positive. Output format: A table with columns: Test Case ID, Scenario, Pre-condition, Test Steps, Test Data, Expected Result, Priority (High/Medium/Low), Test Type (Positive/Negative/Boundary/Performance). Group cases under headings: Keyword Search,

Filters, Sorting, Auto-suggest, Pagination, Invalid/Empty Input, Performance. Minimum 25 test cases. Tone: Professional, precise and technical — clear step-by-step language that a junior tester can execute without clarification.

Scenario 2 — File Upload

✗ **Weak Prompt:** *"Generate test cases for file upload."*

What's missing: No file types, no size limits, no success/failure criteria, no security angle (malicious file upload).

Improved Prompt (ICEPOT)

You are a QA Security Tester with experience in healthcare applications and OWASP file-upload vulnerabilities. Write detailed test cases for the document upload feature covering valid uploads, file-type and size validation, multiple uploads, error handling and security (malicious file) scenarios. Context: The feature is the "Upload Medical Reports" section of a hospital patient portal (web). Allowed types: PDF, JPG, JPEG, PNG, DOCX. Maximum size: 5 MB per file; up to 5 files per upload. Upload via Browse button and drag-and-drop. On success: green toast "File uploaded successfully", file appears in the list with name, size and date, and a progress bar is shown during upload. On failure: inline errors such as "Unsupported file type" or "File exceeds 5 MB limit". Security: executables (.exe, .bat, .sh), double-extension files (report.pdf.exe), files with renamed extensions, and files containing malware (EICAR test file) must be rejected; file names with scripts must be sanitised. Here is an example of the expected test case: TC_UPL_004 | Upload file larger than 5 MB | Steps: 1. Click Browse 2. Select scan_6MB.pdf 3. Click Upload | Test Data: 6 MB PDF | Expected: Upload blocked, error "File exceeds 5 MB limit" displayed, no file added to the list | Priority: High | Type: Negative/Boundary. Output format: A table with columns: Test Case ID, Test Scenario, Test Steps, Test Data (file name/type/size), Expected Result, Priority, Test Type (Functional/Negative/Boundary/Security). Include boundary values (4.99 MB, 5 MB, 5.01 MB, 0 KB). Minimum 20 test cases. Tone: Formal, detail-oriented and security-conscious.

Scenario 3 — Shopping Cart

✗ **Weak Prompt:** *"Test the cart functionality."*

What's missing: No add/remove/update actions, no price calculation validation, no coupon/discount scenarios, no out-of-stock handling.

Improved Prompt (ICEPOT)

You are an experienced E-commerce QA Analyst who specialises in pricing, calculation and checkout validation. Create end-to-end test cases for the shopping cart, validating add, remove and update-quantity actions, price calculations, coupon/discount application and out-of-stock handling. Context: Online grocery web and mobile app. Cart rules: quantity per item 1–10; subtotal = $\Sigma(\text{unit price} \times \text{qty})$; GST 5% applied on subtotal after discount; delivery fee ₹40 for orders below ₹500, free at ₹500 and above. Coupons: SAVE10 (10% off, max ₹100, min order ₹300), FLAT50 (₹50 off, min order ₹250), expired coupon OLD20. Only one coupon per order. If an item goes out of stock while in the cart, show "Out of stock" label and block checkout for that item. Cart persists for logged-in users across sessions; guest cart merges on login. Here is an example of the expected test case: TC_CART_010 | Apply

SAVE10 on ₹1,500 order | Steps: 1. Add items worth ₹1,500 2. Enter SAVE10 3. Click Apply | Expected: Discount = ₹100 (capped, not ₹150), GST 5% on ₹1,400 = ₹70, Delivery ₹0, Total = ₹1,470 | Priority: High | Type: Positive. Output format: A table with columns: Test Case ID, Module (Add/Update/Remove/Price/Coupon/Stock/Persistence), Scenario, Steps, Test Data, Expected Result (with calculated values), Priority, Type. Show the full calculation in the Expected Result for every price-related case. Minimum 25 test cases. Tone: Clear, analytical and numbers-focused.

Scenario 4 — Role-Based Access Control

✗ **Weak Prompt:** *"Write test cases for user roles."*

What's missing: No role types (Admin, Manager, Viewer), no permission matrix, no unauthorized access scenarios, no app domain.

Improved Prompt (ICEPOT)

You are a QA Lead specialising in access-control and security testing of ERP and SaaS applications. Generate RBAC test cases that verify each role can perform only its permitted actions, and that unauthorized access attempts (UI and direct URL/API) are blocked. Context: Domain: an ERP accounting application (Invoice module). Roles: Admin – Create, View, Edit, Delete, Approve invoices, and manage users/roles. Manager – Create, View, Edit, Approve invoices; cannot Delete or manage users. Viewer – View only; cannot Create, Edit, Delete or Approve. Unauthorized actions must be hidden or disabled in the UI; direct URL access (e.g. /invoice/edit/101) must show "403 – Access Denied". Permission changes by Admin must take effect on the user's next login. Deactivated users cannot log in. Here is an example of the expected test case: TC_RBAC_012 | Viewer tries to edit invoice via direct URL | Pre-condition: logged in as Viewer | Steps: 1. Paste /invoice/edit/101 in the address bar | Expected: 403 Access Denied page, invoice not editable, attempt logged in audit trail | Priority: High | Type: Negative/Security. Output format: First, a Permission Matrix table (Roles × Actions, with ✓/✗). Then a test case table with columns: Test Case ID, Role, Action, Access Method (UI/URL/API), Steps, Expected Result, Priority, Type (Positive/Negative). Cover every role–action cell of the matrix. Minimum 20 test cases. Tone: Structured, precise and security-focused.

Scenario 5 — API Testing

✗ **Weak Prompt:** *"Write API test cases for user registration."*

What's missing: No HTTP method, no request/response structure, no status codes to validate, no tool context (Postman/RestAssured).

Improved Prompt (ICEPOT)

You are an API Test Engineer experienced with Postman, RestAssured and REST standards. Write API test cases for the user registration endpoint covering valid requests, field validations, duplicate users, invalid payloads, headers, status codes and response schema. Context: Endpoint: POST https://api.example.com/v1/users/register. Header: Content-Type: application/json. Request body: { "firstName": "Priya", "lastName": "K", "email":

"priya@test.com", "password": "Test@1234", "phone": "9876543210" }. Rules: all fields mandatory; email must be valid and unique; password 8–16 chars with 1 uppercase, 1 number, 1 special char; phone 10 digits. Responses: 201 Created with { "userId", "email", "createdAt" }; 400 Bad Request for validation errors with an error message per field; 409 Conflict for duplicate email; 415 for wrong content type; 405 for wrong method. Response time < 1000 ms. Tool: Postman (with test scripts); automation later in RestAssured. Here is an example of the expected test case: TC_API_005 | Register with already existing email | Method: POST | Body: email = priya@test.com (existing) | Expected: Status 409, body { "error": "Email already registered" }, response time < 1000 ms | Postman test: pm.response.to.have.status(409). Output format: A table with columns: Test Case ID, Scenario, HTTP Method, Endpoint, Headers, Request Body, Expected Status Code, Expected Response, Validation Points. After the table, give sample Postman test scripts for 3 key cases. Minimum 20 test cases. Tone: Technical, concise and developer-friendly.

Scenario 6 — Mobile App (OTP Login)

✗ **Weak Prompt:** "Test OTP login on mobile."

What's missing: No platform (Android/iOS), no OTP expiry time, no resend behavior, no network failure scenario, no Appium/device context.

Improved Prompt (ICEPOT)

You are a Mobile QA Engineer experienced in manual and Appium-based testing of Android and iOS fintech apps. Generate test cases for mobile-number OTP login covering OTP generation, validation, expiry, resend, attempt limits, network interruptions and device/platform-specific behaviour. Context: A banking mobile app on Android 11–14 and iOS 16–17. User enters a 10-digit mobile number and receives a 6-digit OTP via SMS. OTP valid for 2 minutes. Resend enabled after 30 seconds (countdown shown), max 3 resends per session. Max 3 wrong attempts → account locked for 15 minutes. Android supports SMS auto-read; iOS supports OTP autofill from keyboard. Scenarios must include airplane mode, switching Wi-Fi ↔ mobile data, slow 2G, app backgrounded and killed during OTP entry, and incoming calls. Test devices: Samsung Galaxy S23, Redmi Note 12, iPhone 13, iPhone 15. Automation with Appium is planned. Here is an example of the expected test case: TC_OTP_007 | Enter OTP after expiry | Steps: 1. Enter valid mobile number 2. Wait 2 min 10 sec 3. Enter received OTP 4. Tap Verify | Expected: Error "OTP expired. Please request a new OTP", login not allowed, Resend option active | Platform: Android & iOS | Priority: High | Type: Negative. Output format: A table with columns: Test Case ID, Scenario, Platform (Android/iOS/Both), Device/OS, Pre-condition, Steps, Expected Result, Priority, Automatable with Appium (Yes/No). Group by: OTP Generation, Validation, Expiry, Resend, Lockout, Network, Interruptions. Minimum 25 test cases. Tone: Practical, precise and user-experience aware.

Scenario 7 — Dashboard / Report Generation

✗ **Weak Prompt:** "Test the reports section."

What's missing: No report type (PDF/Excel/CSV), no date range filters, no data accuracy validation, no permission-based visibility.

Improved Prompt (ICEPOT)

You are a QA Analyst specialising in reporting, BI dashboards and data validation. Create test cases for the Sales Report dashboard covering report generation, date-range filters, export formats, data accuracy against the source data, and role-based visibility. Context: Web-based sales management system. Reports: Daily Sales, Monthly Revenue, Top 10 Products, Region-wise Sales. Date filters: Today, Last 7 Days, This Month, Custom Range (max 1 year; From date cannot be after To date; future dates not allowed). Export formats: PDF, Excel (.xlsx), CSV — file name format SalesReport_<type>_<DDMMYYYY>.ext. Dashboard totals must match the database/transaction records exactly (amounts to 2 decimals, INR). Visibility: Admin – all regions and export; Regional Manager – own region only, export allowed; Sales Executive – own sales only, no export. Report with no data shows "No records found". Reports up to 10,000 rows must load within 5 seconds. Here is an example of the expected test case: TC_RPT_009 | Validate Monthly Revenue total for March 2026 | Steps: 1. Select Monthly Revenue 2. Choose Custom Range 01/03/2026–31/03/2026 3. Click Generate | Expected: Total revenue on dashboard equals SUM(amount) from orders table for March 2026; same value appears in exported PDF, Excel and CSV | Priority: High | Type: Data Accuracy. Output format: A table with columns: Test Case ID, Report Type, Scenario, Role, Filter/Test Data, Steps, Expected Result, Priority, Type (Functional/Negative/Data Accuracy/Permission/Export/Performance). Minimum 25 test cases. Tone: Analytical, accurate and business-oriented.